

Modelos de Clasificación

Sesión 10

Agenda

- **1.** Modelos de Clasificación
- **2.** Métricas para Clasificación
- **3.** Regresión Logística
- **4.** Árboles de Decisión
- **5.** Clasificación con Spark MLlib

PARTE 1

Modelos de Clasificación

Conceptos fundamentales del aprendizaje supervisado

¿Qué es la clasificación?

Definición y tipos

La **clasificación** es una tarea de aprendizaje supervisado donde el objetivo es predecir una **etiqueta de clase discreta** para cada observación a partir de variables de entrada (Hastie et al., 2009).

Tipos por número de clases

- **Binaria:** dos clases (spam / no spam)
- **Multiclase:** tres o más clases (especie de flor)
- **Multi-etiqueta:** múltiples etiquetas simultáneas (categorías de artículo)

Proceso general

1. Conjunto de datos etiquetado
2. Entrenamiento del modelo
3. Predicción sobre nuevos datos
4. Evaluación con métricas

El modelo aprende una función $f: X \rightarrow Y$ que mapea características a clases (Hastie et al., 2009).

Clasificadores probabilísticos vs. Hard classifiers

Umbral de decisión

Clasificadores probabilísticos

- Producen una **probabilidad** $P(y = 1 | x)$
- Permiten ajustar el umbral de decisión
- Ejemplos: regresión logística, Naive Bayes
- Más flexibles ante desbalance de clases

Hard classifiers

- Producen directamente una **etiqueta**
- El umbral está fijo (por defecto 0.5)
- Ejemplos: SVM con kernel, k-NN
- Más simples de interpretar

Elegir el **umbral** correcto depende del costo relativo de los errores de tipo I y tipo II en cada problema (James et al., 2013).

Desbalance de clases

Oversampling, undersampling y SMOTE

En muchos problemas reales, las clases no están distribuidas uniformemente (James et al., 2013).

Técnica	Descripción	Ventaja
Undersampling	Reducir muestras de la clase mayoritaria	Rápido, menos datos
Oversampling	Duplicar muestras de la clase minoritaria	Sin pérdida de información
SMOTE	Generar muestras sintéticas interpolando vecinos	Más diversidad sintética
Pesos de clase	Penalizar más los errores en la clase rara	Sin modificar datos

SMOTE (Synthetic Minority Over-sampling Technique) genera nuevas instancias interpolando entre muestras existentes de la clase minoritaria, evitando la simple duplicación.

Aplicaciones reales

¿Dónde se usa la clasificación?

La clasificación está presente en prácticamente todas las industrias (Goodfellow et al., 2016).

Ejemplos por dominio

- **Finanzas:** detección de fraude en transacciones
- **Salud:** diagnóstico médico (tumor benigno/maligno)
- **Marketing:** predicción de churn (abandono)

Criterios de elección del modelo

- Volumen y dimensionalidad de los datos
- Interpretabilidad requerida
- Velocidad de predicción
- Distribución de clases
- Costo de errores (FP vs. FN)

PARTE 2

Métricas para Clasificación

Cómo medir el rendimiento de un modelo

Matriz de Confusión

La base de todas las métricas

La **matriz de confusión** resume los resultados de predicción de un clasificador binario (Powers, 2011).

	Predicho: Positivo	Predicho: Negativo
Real: Positivo	TP (Verdadero Positivo)	FN (Falso Negativo)
Real: Negativo	FP (Falso Positivo)	TN (Verdadero Negativo)

- **TP:** modelo dice positivo, es positivo
- **TN:** modelo dice negativo, es negativo
- **FP:** modelo dice positivo, es negativo (Error tipo I)
- **FN:** modelo dice negativo, es positivo (Error tipo II)

Todos los errores no tienen el mismo costo. En diagnóstico médico, un **FN** (enfermedad no detectada) es mucho más grave que un **FP** (Powers, 2011).

Accuracy, Precision, Recall y Specificity

Las métricas fundamentales

Derivadas directamente de la matriz de confusión (Powers, 2011):

Métrica	Fórmula	Interpreta
Accuracy	$(TP + TN) / \text{Total}$	% de predicciones correctas
Precision	$TP / (TP + FP)$	De los que predije positivos, ¿cuántos lo eran?
Recall	$TP / (TP + FN)$	De los positivos reales, ¿cuántos detecté?
Specificity	$TN / (TN + FP)$	De los negativos reales, ¿cuántos identifiqué?

Accuracy puede ser engañosa con clases desbalanceadas. Un modelo que predice siempre la clase mayoritaria puede tener 99% de accuracy pero ser inútil (Scikit-learn developers, n.d.).

F1-Score

Media armónica entre Precision y Recall

El **F1-Score** equilibra la precisión y el recall en una sola métrica (Powers, 2011):

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

¿Cuándo usar F1?

- Clases desbalanceadas
- Ambos errores (FP y FN) son importantes
- Comparar modelos con trade-off diferente

Variantes

- **F-beta**: prioriza recall ($\beta > 1$) o precision ($\beta < 1$)
- **Macro F1**: promedio no ponderado por clase
- **Weighted F1**: promedio ponderado por soporte

Si el costo de FN $>$ FP, usar **recall** como métrica principal. Si FP $>$ FN, usar **precision**. F1 balancea ambas.

Curva ROC y AUC

Evaluación independiente del umbral

La **curva ROC** (Receiver Operating Characteristic) grafica TPR vs. FPR para todos los umbrales posibles (Fawcett, 2006).

Componentes

- **TPR (Recall):** $TP / (TP + FN)$
- **FPR:** $FP / (FP + TN)$
- **AUC:** Área bajo la curva ROC

Interpretación del AUC

- $AUC = 1.0 \rightarrow$ clasificador perfecto
- $AUC = 0.5 \rightarrow$ clasificador aleatorio
- $AUC < 0.5 \rightarrow$ peor que aleatorio

Ventajas de ROC-AUC

- No depende del umbral elegido
- Útil para comparar modelos
- Robusto ante desbalance moderado

Alternativa: PR Curve

- Mejor para desbalance severo
- Grafica Precision vs. Recall

Fawcett (2006) propone ROC-AUC como métrica estándar para comparación de clasificadores binarios.

Métricas para Datasets Desbalanceados

Evaluación con distribuciones asimétricas

Cuando las clases están muy desbalanceadas, las métricas estándar pueden ser engañosas (Scikit-learn developers, n.d.).

Situación	Métrica recomendada	Razón
Desbalance moderado	F1 Weighted	Pondera por soporte de cada clase
Desbalance severo	PR-AUC	Más informativa que ROC-AUC
Costo asimétrico	Recall o Precision	Según qué error es más costoso
Comparación general	Cohen's Kappa	Corrige por azar

Cohen's Kappa (κ) mide el acuerdo entre predicciones y etiquetas reales corrigiendo por la probabilidad de acuerdo al azar. $\kappa > 0.8$ indica excelente concordancia.

PARTE 3

Regresión Logística

Del espacio lineal a la probabilidad de clase

Función Sigmoides

El corazón de la regresión logística

La **regresión logística** transforma una combinación lineal de características en una probabilidad usando la función sigmoide (Bishop, 2006):

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \text{donde} \quad z = \mathbf{w}^T \mathbf{x} + b$$

Propiedades de $\sigma(z)$

- Salida siempre en $(0, 1)$
- $\sigma(0) = 0.5$
- Monótonamente creciente
- Diferenciable en todo su dominio

Decisión binaria

- $P(y=1|x) = \sigma(z)$
- $P(y=0|x) = 1 - \sigma(z)$
- Predice clase 1 si $\sigma(z) \geq 0.5$
- Equivalente a $z \geq 0$

A pesar del nombre, la regresión logística es un **modelo de clasificación**, no de regresión (Bishop, 2006).

Frontera de Decisión

Separación lineal en el espacio de características

La frontera de decisión es el conjunto de puntos donde $P(y=1|x) = 0.5$, es decir, donde $z = 0$

(Ng, n.d.):

$$\mathbf{w}^T \mathbf{x} + b = 0$$

Características

- La frontera es **lineal** en el espacio original
- En 2D: una línea recta
- En nD: un hiperplano de dimensión $n-1$
- No captura relaciones no lineales directamente

Extensiones no lineales

- Agregar términos polinomiales: x^2 , $x_1 \cdot x_2$
- Transformaciones de características
- Kernels (SVM logístico)
- Redes neuronales

Para fronteras no lineales con regresión logística, se pueden agregar **features polinomiales** antes de entrenar (Ng, n.d.).

Estimación de Máxima Verosimilitud (MLE)

Cómo se ajustan los parámetros

Los parámetros **w** se estiman maximizando la verosimilitud de los datos observados (Murphy, 2012):

$$\mathcal{L}(\mathbf{w}) = \prod_{i=1}^n P(y_i | \mathbf{x}_i; \mathbf{w})$$

Equivalentemente, se minimiza la **log-loss** (entropía cruzada binaria):

$$J(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

Optimización

- No tiene solución cerrada
- Se usa **Gradient Descent**
- Variantes: SGD, L-BFGS, Newton-CG

Convergencia

- Función convexa → mínimo global garantizado
- Sensible a la escala de features
- Normalizar variables antes de entrenar

Regularización

Controlar el sobreajuste

La regularización añade una penalización sobre los coeficientes para evitar el overfitting (Bishop, 2006):

L2 — Ridge

$$J_{L2} = J + \lambda \sum_j w_j^2$$

- Encoge coeficientes hacia cero
- No los hace exactamente cero
- Estable ante multicolinealidad
- Optimizer: L-BFGS

L1 — Lasso

$$J_{L1} = J + \lambda \sum_j |w_j|$$

- Puede fijar coeficientes en cero
- Selección implícita de features
- Útil en alta dimensionalidad
- Optimizer: coordenada descendente

Elastic Net combina L1 y L2: penaliza con $\alpha \cdot L1 + (1-\alpha) \cdot L2$. Útil cuando hay muchas features correlacionadas (Bishop, 2006).

PARTE 4

Árboles de Decisión

Particionando el espacio de características recursivamente

¿Qué es un Árbol de Decisión?

Estructura y funcionamiento

Un **árbol de decisión** particiona el espacio de características en regiones rectangulares y asigna una clase a cada región (Breiman et al., 1984).

Componentes del árbol

- **Nodo raíz:** primera partición (feature más informativa)
- **Nodos internos:** condiciones de división ($x \leq \text{umbral}$)
- **Ramas:** resultado de cada condición (sí/no)
- **Hojas:** predicción final (clase mayoritaria)

Ventajas

- Interpretable y visualizable
- No requiere normalización de datos
- Maneja features numéricas y categóricas
- Captura relaciones no lineales

Desventajas

- Tendencia al sobreajuste
- Alta varianza (inestable)
- Fronteras solo ortogonales

Algoritmo ID3 — Entropía e Information Gain

Selección de la mejor división

El algoritmo **ID3** elige en cada nodo la feature que maximiza el **Information Gain** (Quinlan, 1993):

$$\text{Entropía}(S) = - \sum_c p_c \log_2 p_c$$

$$\text{IG}(S, A) = H(S) - \sum_{v \in \text{valores}(A)} \frac{|S_v|}{|S|} H(S_v)$$

Interpretación

- Entropía = 0: nodo puro (una sola clase)
- Entropía = 1: máxima impureza (50/50)
- IG mide cuánto reduce la entropía dividir por A

Limitaciones de ID3

- Sesgo hacia features con muchos valores
- No soporta valores continuos directamente
- Sin mecanismo de poda
- C4.5 corrige estas limitaciones (Quinlan, 1993)

CART — Impureza Gini

La alternativa a la entropía

CART (Classification and Regression Trees) usa la **impureza Gini** como criterio de división (Breiman et al., 1984):

$$\text{Gini}(S) = 1 - \sum_c p_c^2$$
$$\text{Gini}(S, A) = \sum_v \frac{|S_v|}{|S|} \cdot \text{Gini}(S_v)$$

Gini vs. Entropía

Criterio	Cálculo	Velocidad
Gini	Sin logaritmo	Más rápido
Entropía	Con $\log_2$	Ligeramente más lento
Resultado	Similar en práctica	—

Características de CART

- Solo particiones **binarias**
- Soporta regresión y clasificación
- Base de Random Forests y Gradient Boosting
- Incluye cost-complexity pruning (Breiman et al., 1984)

Partición Recursiva

Construcción del árbol paso a paso

El árbol se construye con un algoritmo **greedy top-down** (Breiman et al., 1984):

```

función ConstruirÁrbol(S, features):
    si S es puro o criterio de parada → retornar hoja con clase mayoritaria

    para cada feature A y umbral t:
        calcular IG(S, A, t)

    (A*, t*) = argmax IG
    S_izq = {x ∈ S : x[A*] ≤ t*}
    S_der = {x ∈ S : x[A*] > t*}

    retornar Nodo(A*, t*,
                  ConstruirÁrbol(S_izq),
                  ConstruirÁrbol(S_der))

```

La búsqueda es **greedy**: elige la mejor división local sin garantizar el árbol globalmente óptimo. Esto puede llevar a sobreajuste si no se controla la profundidad (Breiman et al., 1984).

Poda del Árbol

Evitando el sobreajuste

Un árbol sin restricciones memoriza los datos de entrenamiento. La **poda** reduce su complejidad (Quinlan, 1993):

Pre-poda (criterios de parada)

- `max_depth` : profundidad máxima
- `min_samples_split` : mínimo de muestras para dividir
- `min_samples_leaf` : mínimo en nodos hoja
- `max_features` : features a considerar

Post-poda

- **Cost-complexity pruning (α)**: elimina subárboles que no mejoran lo suficiente la precisión ponderada por tamaño
- **Reduced error pruning**: elimina nodos si el error en validación no aumenta
- Selección de α con cross-validation

En Spark MLlib, `maxDepth` es el parámetro más importante. Valores típicos: 3–8. Un árbol con profundidad > 10 casi siempre tiene sobreajuste.

PARTE 5

Clasificación con Spark MLlib

Pipelines distribuidos para datos a escala

Pipeline API de Spark MLlib

Stages, Transformers y Estimators

La **Pipeline API** de Spark MLlib organiza el flujo de ML en etapas encadenadas y reproducibles (Apache Software Foundation, n.d.):

Transformer

- Aplica una transformación a un DataFrame
- Método: `.transform(df)`
- Ejemplos: `StringIndexer`,
`VectorAssembler`, `StandardScaler`
- No aprende parámetros del dato

Estimator

- Aprende parámetros del dato de entrenamiento
- Método: `.fit(df)` → retorna un **Model** (Transformer)
- Ejemplos: `LogisticRegression`,
`DecisionTreeClassifier`
- El Model resultante puede transformar nuevos datos

```
from pyspark.ml import Pipeline

pipeline = Pipeline(stages=[indexer, assembler, scaler, classifier])
model = pipeline.fit(train_df)
predictions = model.transform(test_df)
```

Apache Software Foundation (n.d.)

Feature Engineering en Spark MLlib

Preparación de datos para el modelo

Spark MLlib provee transformers estándar para preparar las features (Apache Software Foundation, n.d.):

```
from pyspark.ml.feature import StringIndexer, VectorAssembler, StandardScaler

# Codificar variables categóricas
indexer = StringIndexer(inputCol="categoria", outputCol="categoria_idx")

# Combinar features en un vector
assembler = VectorAssembler(
    inputCols=["edad", "ingreso", "categoria_idx"],
    outputCol="features_raw"
)

# Normalizar (media=0, desv=1)
scaler = StandardScaler(inputCol="features_raw", outputCol="features")
```

Transformer	Propósito
<code>StringIndexer</code>	Categoría → índice numérico
<code>VectorAssembler</code>	Columnas → vector de features
<code>StandardScaler</code>	Normalización Z-score
<code>OneHotEncoder</code>	Índice → vector binario

LogisticRegression en Spark MLlib

Clasificación logística distribuida

Spark MLlib ofrece una implementación distribuida y escalable de regresión logística (Karau & Warren, 2015):

```
from pyspark.ml.classification import LogisticRegression

lr = LogisticRegression(
    featuresCol="features",
    labelCol="label",
    maxIter=100,
    regParam=0.01,          #  $\lambda$  de regularización
    elasticNetParam=0.0,    # 0=L2, 1=L1
    family="binomial"      # "multinomial" para multiclase
)

lr_model = lr.fit(train_df)
print(lr_model.coefficients)
print(lr_model.intercept)
```

DecisionTreeClassifier en Spark MLlib

Árbol de decisión distribuido

Spark MLlib implementa CART con soporte nativo para datos distribuidos (Apache Software Foundation, n.d.):

```
from pyspark.ml.classification import DecisionTreeClassifier

dt = DecisionTreeClassifier(
    featuresCol="features",
    labelCol="label",
    maxDepth=5,                # profundidad máxima
    minInstancesPerNode=10,    # pre-poda por tamaño
    impurity="gini"            # "entropy" también disponible
)

dt_model = dt.fit(train_df)

# Ver la estructura del árbol
print(dt_model.toDebugString)

# Importancia de features
print(dt_model.featureImportances)
```

`featureImportances` devuelve un vector sparse con la importancia de cada feature basada en la **reducción total de impureza** ponderada por el número de muestras (Apache Software Foundation, n.d.).

Evaluadores en Spark MLlib

Medir el rendimiento del modelo

Spark MLlib provee evaluadores estándar para modelos de clasificación (Zaharia et al., 2016):

```
from pyspark.ml.evaluation import (
    BinaryClassificationEvaluator,
    MulticlassClassificationEvaluator
)

# Para clasificación binaria
binary_eval = BinaryClassificationEvaluator(
    labelCol="label",
    rawPredictionCol="rawPrediction",
    metricName="areaUnderROC"    # o "areaUnderPR"
)

# Para clasificación multiclase
multi_eval = MulticlassClassificationEvaluator(
    labelCol="label",
    predictionCol="prediction",
    metricName="f1"    # "accuracy", "weightedPrecision", "weightedRecall"
)

auc = binary_eval.evaluate(predictions)
f1  = multi_eval.evaluate(predictions)
```

CrossValidator y ParamGridBuilder

Ajuste de hiperparámetros distribuido

Spark MLlib permite realizar **k-fold cross-validation** de forma distribuida sobre todo el pipeline (Karau & Warren, 2015):


```
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder

# Grilla de hiperparámetros
param_grid = (ParamGridBuilder()
    .addGrid(lr.regParam,      [0.01, 0.1, 1.0])
    .addGrid(lr.elasticNetParam, [0.0, 0.5, 1.0])
    .addGrid(dt.maxDepth,      [3, 5, 8])
    .build())

cv = CrossValidator(
    estimator=pipeline,
    estimatorParamMaps=param_grid,
    evaluator=binary_eval,
    numFolds=5,
    parallelism=4    # evaluaciones paralelas
)

cv_model = cv.fit(train_df)
best_model = cv_model.bestModel
```

Flujo Completo: Pipeline de Clasificación en Spark

Integrando todos los componentes

```

from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, VectorAssembler, StandardScaler
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import BinaryClassificationEvaluator
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder

# 1. Preparación
indexer = StringIndexer(inputCol="cat", outputCol="cat_idx")
assembler = VectorAssembler(inputCols=["f1", "f2", "cat_idx"], outputCol="feats_raw")
scaler = StandardScaler(inputCol="feats_raw", outputCol="features")

# 2. Modelo
lr = LogisticRegression(featuresCol="features", labelCol="label")

# 3. Pipeline + CV
pipeline = Pipeline(stages=[indexer, assembler, scaler, lr])
param_grid = ParamGridBuilder().addGrid(lr.regParam, [0.01, 0.1]).build()
cv = CrossValidator(estimator=pipeline, estimatorParamMaps=param_grid,
                    evaluator=BinaryClassificationEvaluator(), numFolds=5)

# 4. Entrenamiento y evaluación
cv_model = cv.fit(train_df)
predictions = cv_model.transform(test_df)

```

Referencias

Referencias

Apache Software Foundation. (n.d.). *MLlib: Machine learning library*.

<https://spark.apache.org/docs/latest/ml-guide.html>

Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.

Breiman, L., Friedman, J., Stone, C. J., & Olshen, R. A. (1984). *Classification and regression trees*. Chapman & Hall.

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874. <https://doi.org/10.1016/j.patrec.2005.10.010>

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer. <https://doi.org/10.1007/978-0-387-84858-7>

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An introduction to statistical learning: With applications in R*. Springer. <https://doi.org/10.1007/978-1-4614-7138-7>

Karau, H., & Warren, R. (2015). *Learning Spark: Lightning-fast big data analysis*. O'Reilly Media.

Murphy, K. P. (2012). *Machine learning: A probabilistic perspective*. MIT Press.

Ng, A. (n.d.). *CS229 lecture notes: Supervised learning*. Stanford University.
<https://cs229.stanford.edu/notes/cs229-notes1.pdf>

Powers, D. M. W. (2011). Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation. *Journal of Machine Learning Technologies*, 2(1), 37–63.

Quinlan, J. R. (1993). *C4.5: Programs for machine learning*. Morgan Kaufmann.

Salzberg, S. L. (1994). C4.5: Programs for machine learning [Review]. *Machine Learning*, 16, 235–240. <https://doi.org/10.1007/BF00993309>

Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., & Stoica, I. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11), 56–65. <https://doi.org/10.1145/2934664>